Lab 10 — La Voix du Consommateur : Analyse NLP

Dataset `amazon_review.csv` embarqué directement — aucun fichier à importer !

Installation des dépendances & Chargement du Dataset

```
# ── Installations ──────────────────────────────────────────
!pip install -q textblob wordcloud matplotlib

import io, re, string
from collections import Counter

import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.colors as mcolors
from textblob import TextBlob
import nltk

for pkg in ['punkt', 'punkt_tab', 'stopwords', 'averaged_perceptron_tagger',
            'averaged_perceptron_tagger_eng', 'wordnet', 'omw-1.4']:
    nltk.download(pkg, quiet=True)

from nltk.corpus import stopwords
from nltk.stem import WordNetLemmatizer
from nltk import pos_tag, word_tokenize
from wordcloud import WordCloud

# ── Dataset Amazon Reviews embarqué directement ────────────
_CSV = io.StringIO('reviewText,Positive\n"I love this game, so much fun for the whole family! Will recommend to friends.",1
df = pd.read_csv(_CSV)
print(f"Dataset chargé : {len(df)} avis")
print(df['Positive'].value_counts())
df.head()
```

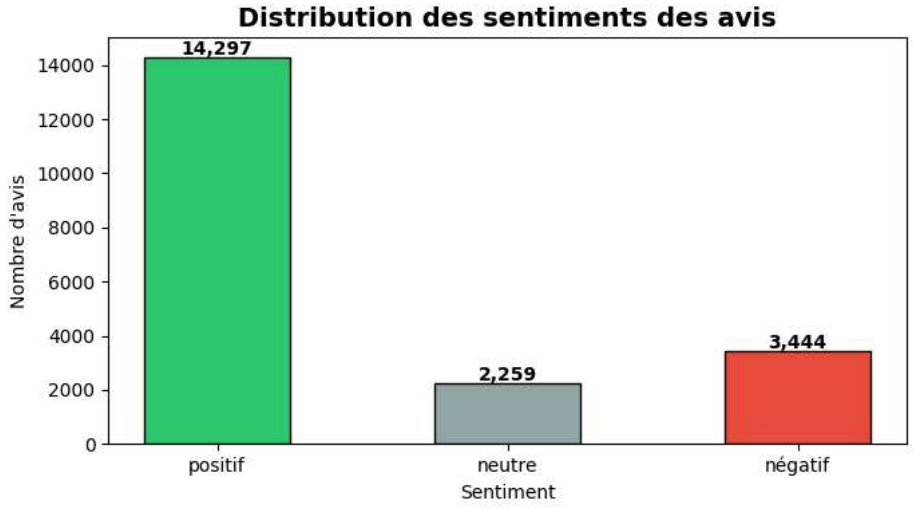
```
Dataset chargé : 20000 avis
Positive
1    14421
0     5579
Name: count, dtype: int64
```

	reviewText	Positive
0	I love this game, so much fun for the whole fa...	1
1	Great product, works perfectly on my device. H...	1
2	Perfect game for my tablet, smooth and enjoyab...	1
3	Not bad, but there is definitely room for impr...	1
4	Very satisfied with this product. Great value ...	1

Tâche 1 : Analyse de sentiment & graphique à barres

```
def get_sentiment(text):
    polarity = TextBlob(str(text)).sentiment.polarity
    if polarity > 0.1:
        return 'positif'
    elif polarity < -0.1:
        return 'négatif'
    else:
        return 'neutre'

df['sentiment'] = df['reviewText'].apply(get_sentiment)

# Distribution
counts = df['sentiment'].value_counts().reindex(['positif', 'neutre', 'négatif'])

fig, ax = plt.subplots(figsize=(7, 4))
bars = ax.bar(counts.index, counts.values,
              color=['#2ecc71', '#95a5a6', '#e74c3c'], edgecolor='black', width=0.5)
ax.set_title("Distribution des sentiments des avis", fontsize=14, fontweight='bold')
ax.set_xlabel("Sentiment")
ax.set_ylabel("Nombre d'avis")
for bar in bars:
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 80,
```

```
            f"{bar.get_height():,}", ha='center', fontweight='bold')
plt.tight_layout()
plt.show()

print(counts)
```




```
sentiment
positif    14297
neutre      2259
négatif     3444
Name: count, dtype: int64
```

Tâche 2 : Prétraitement & Lemmatisation

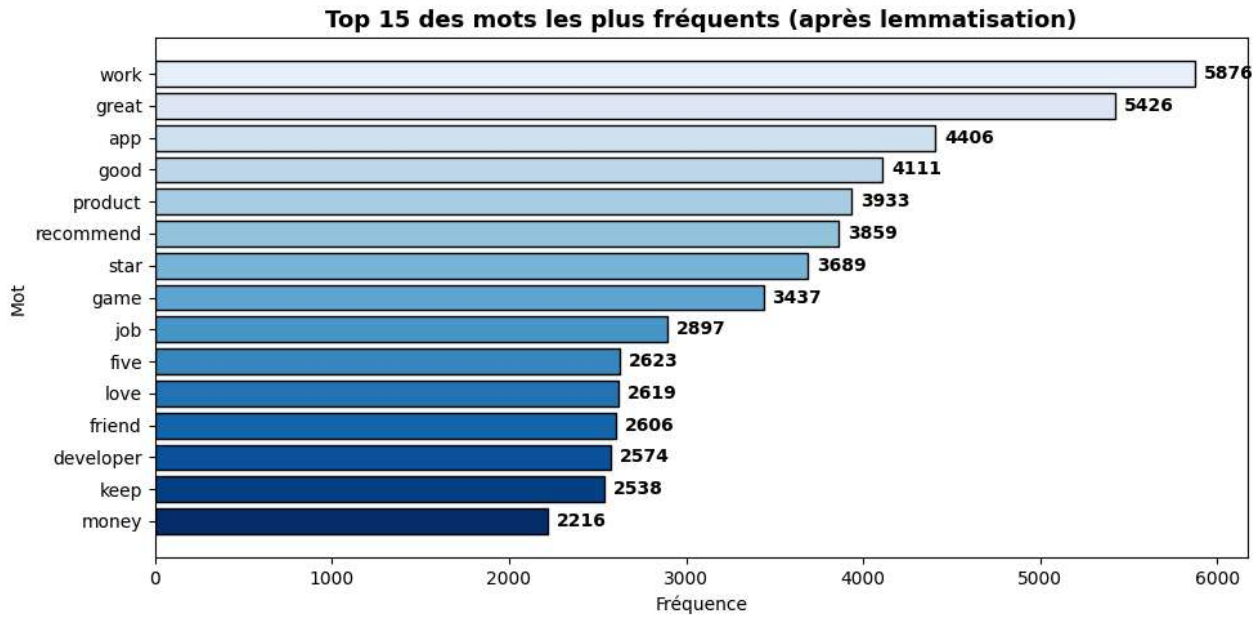
```
STOP_WORDS = set(stopwords.words('english'))
lemmatizer = WordNetLemmatizer()

def get_wordnet_pos(treebank_tag):
    """Convertit les tags POS de NLTK en tags WordNet."""
    from nltk.corpus import wordnet
    if treebank_tag.startswith('J'):
        return wordnet.ADJ
    elif treebank_tag.startswith('V'):
        return wordnet.VERB
    elif treebank_tag.startswith('R'):
        return wordnet.ADV
    else:
        return wordnet.NOUN

def preprocess(text):
    # 1. Minuscules
    text = str(text).lower()
    # 2. Supprimer ponctuation et chiffres
    text = re.sub(r'[^\w\s]', '', text)
    text = re.sub(r'\d+', '', text)
    # 3. Tokenisation
    tokens = word_tokenize(text)
    # 4. Supprimer stopwords et mots courts
    tokens = [t for t in tokens if t not in STOP_WORDS and len(t) > 2]
    # 5. POS tagging + Lemmatisation
    tagged = pos_tag(tokens)
    lemmas = [lemmatizer.lemmatize(word, get_wordnet_pos(tag)) for word, tag in tagged]
    return lemmas

print("Prétraitement en cours...")
df['lemmas'] = df['reviewText'].apply(preprocess)
print(f"Exemple : {df['lemmas'].iloc[0][:10]}")
print("Prétraitement terminé !")
```

```
Prétraitement en cours...
Exemple : ['love', 'game', 'much', 'fun', 'whole', 'family', 'recommend', 'friend']
Prétraitement terminé !
```

Tâche 3 : Top 15 mots les plus fréquents

```
# Aplatir toutes les lemmes en une seule liste
all_words = [word for lemma_list in df['lemmas'] for word in lemma_list]
word_freq = Counter(all_words)
```

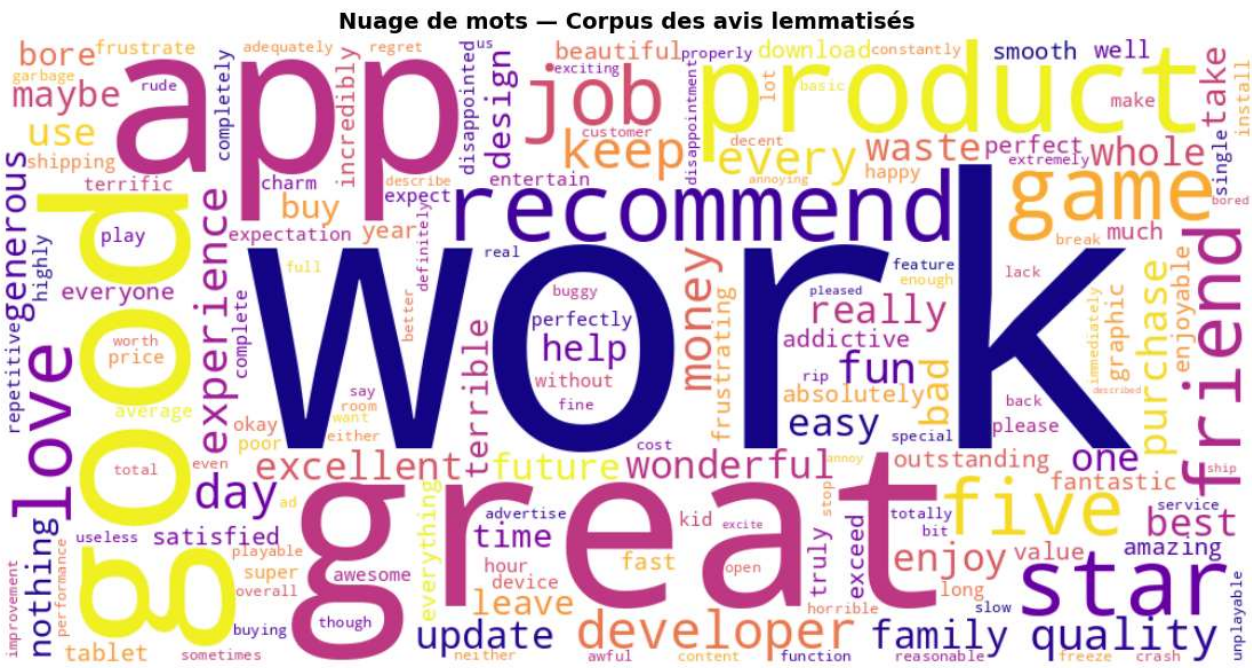
```
top15 = word_freq.most_common(15)

words, freqs = zip(*top15)

fig, ax = plt.subplots(figsize=(10, 5))
bars = ax.barh(list(words)[::-1], list(freqs)[::-1],
               color=plt.cm.Blues_r([i/15 for i in range(15)]),
               edgecolor='black')
ax.set_title("Top 15 des mots les plus fréquents (après lemmatisation)", fontsize=13, fontweight='bold')
ax.set_xlabel("Fréquence")
ax.set_ylabel("Mot")
for bar, freq in zip(bars, list(freqs)[::-1]):
    ax.text(bar.get_width() + 50, bar.get_y() + bar.get_height()/2,
            str(freq), va='center', fontweight='bold')
plt.tight_layout()
plt.show()

print("Top 15 :", top15)
```




```
Top 15 : [('work', 5876), ('great', 5426), ('app', 4406), ('good', 4111), ('product', 3933), ('recommend', 3859), ('star', 3
```

Tâche 4 : Nuage de mots (Word Cloud)

```
text_for_cloud = ' '.join(all_words)

wc = WordCloud(
    width=1000, height=500,
    background_color='white',
    colormap='plasma',
    max_words=200,
    collocations=False
).generate(text_for_cloud)

fig, ax = plt.subplots(figsize=(14, 6))
ax.imshow(wc, interpolation='bilinear')
ax.axis('off')
ax.set_title("Nuage de mots — Corpus des avis lemmatisés", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```



Tâche 5 : Fréquence des noms, verbes et adjectifs

```
# POS tagging sur l'ensemble du corpus
print("Calcul des POS tags en cours...")
all_tokens_tagged = []
for lemma_list in df['lemmas']:
    tagged = pos_tag(lemma_list)
    all_tokens_tagged.extend(tagged)

nouns      = [w for w, t in all_tokens_tagged if t.startswith('NN')]
verbs      = [w for w, t in all_tokens_tagged if t.startswith('VB')]
adjectives = [w for w, t in all_tokens_tagged if t.startswith('JJ')]

pos_counts = {
    'Noms\n(Nouns)': len(nouns),
    'Verbes\n(Verbs)': len(verbs),
    'Adjectifs\n(Adjectives)': len(adjectives)
}

fig, ax = plt.subplots(figsize=(7, 4))
bars = ax.bar(pos_counts.keys(), pos_counts.values(),
              color=['#3498db', '#e67e22', '#9b59b6'],
              edgecolor='black', width=0.45)
ax.set_title("Fréquence des catégories grammaticales", fontsize=13, fontweight='bold')
ax.set_ylabel("Nombre d\'occurrences")
for bar in bars:
    ax.text(bar.get_x() + bar.get_width()/2, bar.get_height() + 200,
            f"{bar.get_height():,}", ha='center', fontweight='bold')
plt.tight_layout()
plt.show()

print(f"Noms : {len(nouns):,}  |  Verbes : {len(verbs):,}  |  Adjectifs : {len(adjectives):,}")
```

```
Calcul des POS tags en cours...
```